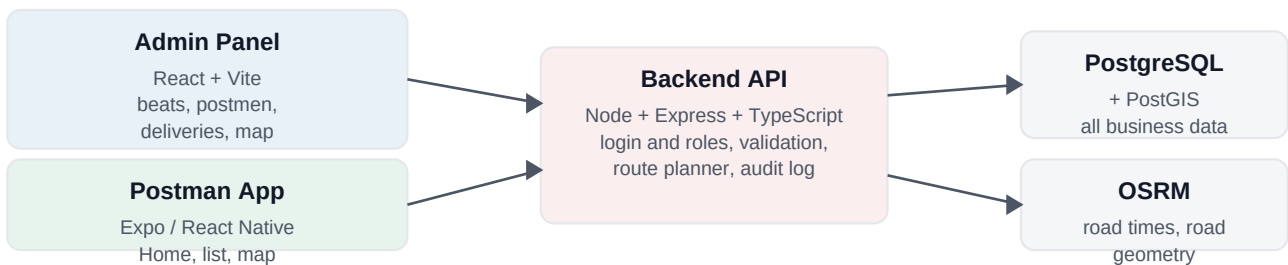


How the system works

Postal Delivery Operations System: the admin panel, the postman app and the backend, in plain words.

1. The big picture

Three programs work together. The **admin panel** is for post-office staff. The **postman app** is for the person on the street. The **backend** sits between them and the database, and is the only place where rules are applied and data is saved. Neither app keeps its own copy of the truth: they ask the backend and show what it says.



2. What each part does

Part	Who uses it	What it does
Admin panel	Administrators	Draws and verifies beats on a map, uploads a beat list, manages postmen (with profile photos), imports and assigns deliveries, resolves exceptions, reads reports and the audit log.
Postman app	Postmen	Home (today at a glance), the delivery list in route order, delivery details, the route map, Navigate, Start Delivery / Mark Delivered, works offline and syncs later.
Backend	Both apps	Login and roles, saving to PostgreSQL/PostGIS, matching parcels to beats, planning routes, storing photos, writing the audit log.
PostgreSQL + PostGIS	Backend only	Stores everything: post offices, beats and their map polygons, postmen, deliveries, history, routes.
OSRM	Backend only	A road-routing service: real driving times between stops and the road line drawn on the map.

3. Life of a parcel

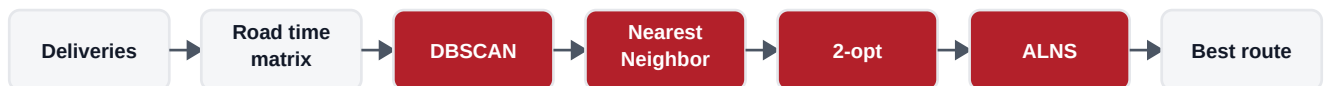
- **Arrives.** An admin adds a delivery, or imports a file. The address is turned into map coordinates.
- **Matched to a beat.** PostGIS checks which *verified* beat polygon contains the address. The beat's postman gets the parcel; if nobody covers it, it appears in Assignment Exceptions for an admin to fix.
- **Routed.** The backend plans the postman's visiting order (section 5) and the app shows it as numbered stops.
- **Delivered.** The postman taps Start Delivery, then Mark Delivered. The backend checks the change is allowed, saves it with history and the audit log, and removes the stop from the route without reshuffling the rest.
- **Problems.** Recipient unavailable, wrong address and similar outcomes re-plan the route and can raise an exception for the admin.

4. Beats and verification

A beat is an area on the map with one postman. An admin uploads a beat list (Excel or CSV) and the panel walks through *Upload*, *Validate*, *Review*, *Import*: nothing is saved until the last step, and every problem (duplicate numbers, missing territories, unreadable boundaries) is shown and can be downloaded as an error report. Imported beats are **Pending verification**; the admin checks each polygon on the map and verifies it. Only verified beats receive parcels. Who verified and when is stored in the database.

5. How the route is chosen

There is exactly one optimizer. It is not a setting, and neither the app nor the admin can change it.



Step	What it does
Road-time matrix	Asks OSRM for the driving time between the start point and every stop. Answers are cached, so repeated plans do not call OSRM again.
DBSCAN	Groups stops that are close <i>by road time</i> into neighbourhoods.
Nearest Neighbor	Builds a first route: neighbourhood by neighbourhood, always going to the closest next stop.
2-opt	Removes crossings by reversing stretches of the route, only when the cost really drops.
ALNS	Repeatedly removes some stops and puts them back in better places (random, worst, related and cluster removal; greedy, regret-2 and regret-3 insertion). Operators that work well are used more often, worse routes are sometimes accepted early on to escape dead ends, and the best route ever seen is what is returned.

Every step is scored with the **same cost**: driving time, plus a penalty for carrying a heavy bag over long legs, plus a penalty for making urgent parcels wait, with time spent at each door included. So a route can only be called better if that number is lower. On real road data ALNS improved the 2-opt route by 7 to 30 percent for 10 to 100 stops, and 100 stops plan in under 3 seconds. When a delivery is completed the route is only trimmed; a new delivery or a manual recalculation runs the whole pipeline again.

6. Login, roles and safety

- Everyone signs in with a login; the backend issues short-lived tokens. A postman's identity always comes from the login, never from anything the app sends.
- **Roles:** Super Admin (all post offices), Admin (their own post office only), Postman (only their own deliveries). Another office's data looks like it does not exist.
- Profile photos are checked by their real file type and size, can only be changed by an admin, and are stored outside the database (only the address is saved).
- Every important change is written to an audit log: who, what, when.
- The backend validates every request itself. The apps' buttons are a convenience, not the protection.

7. Data that survives

Beats, polygons, verification, postmen, photos, assignments, deliveries, statuses and routes are all in PostgreSQL. A browser refresh, a logout/login and a backend restart were each tested to leave them unchanged. The only thing kept on the phone is the queue of updates made while offline; it is sent when the connection returns, and if the server has moved on (for example an admin cancelled the parcel) the phone's stale change is not forced through: the postman is told instead.

8. Running and testing it

What	How
Backend	<code>cd backend</code> then <code>npm run dev</code> (needs PostgreSQL with PostGIS and the <code>.env</code> values). Tests: <code>npm test</code> .
Admin panel	<code>cd frontend</code> then <code>npm run dev</code> (opens on port 5173).
Postman app	<code>cd PostmanApp</code> then <code>npx expo start --web</code> , or open it on a phone with Expo.
Route checks	<code>npm run route:alns</code> measures 1 to 100 stops; <code>npm run route:verify</code> checks a live route end to end.
Road routing	<code>OSRM_BASE_URL</code> in the backend <code>.env</code> . Without it the routes still work but use flagged straight-line estimates.

Known limits: the backend and admin panel have no ESLint set up yet; the public OSRM demo server is for development only (self-host it for real use); local photo storage suits development and should be swapped for object storage in production.